

Accelerating Deep Neural Networks on Mobile Multicore NPUs

Hanwoong Jung
Samsung Advanced Institute
of Technology
Suwon, Korea
hw7884.jung@samsung.com

Hexiang Ji
Samsung R&D Institute
Xian, China
hexiang.ji@samsung.com

Alexey Pushchin
Samsung R&D Institute
Moscow, Russia
a.puschin@samsung.com

Maxim Ostapenko
Samsung Advanced Institute
of Technology
Suwon, Korea
maxim.o@samsung.com

Wenlong Niu
Samsung R&D Institute
Xian, China
wenlong.niu@samsung.com

Ilya Palachev
Samsung R&D Institute
Moscow, Russia
i.palachev@samsung.com

Yutian Qu
Samsung R&D Institute
Xian, China
yutian01.qu@samsung.com

Pavel Fedin
Samsung R&D Institute
Moscow, Russia
p.fedin@samsung.com

Yuri Gribov
Samsung R&D Institute
Moscow, Russia
y.gribov@samsung.com

Heewoo Nam
Samsung Advanced Institute
of Technology
Suwon, Korea
hee-woo.nam@samsung.com

Dongguen Lim
Samsung Advanced Institute
of Technology
Suwon, Korea
dongguen.lim@samsung.com

Hyunjun Kim
Samsung Advanced Institute
of Technology
Suwon, Korea
hjun20.kim@samsung.com

Joonho Song
Samsung Advanced Institute
of Technology
Suwon, Korea
joonho71.song@samsung.com

Seungwon Lee
Samsung Advanced Institute
of Technology
Suwon, Korea
seungw.lee@samsung.com

Hwansoo Han
Sungkyunkwan University
Suwon, Korea
hhan@skku.edu

Abstract

Neural processing units (NPUs) have become indispensable parts of mobile SoCs. Furthermore, integrating multiple NPU cores into a single chip becomes a promising solution for ever-increasing computing power demands in mobile devices. This paper addresses techniques to maximize the utilization of NPU cores and reduce the latency of on-device inference. Mobile NPUs typically have a small amount of local memory (or scratch pad memory, SPM) that provides space only enough for input/output tensors and weights of one layer operation in deep neural networks (DNNs). Even in multicore NPUs, such local memories are distributed across the cores. In such systems, executing network layer operations in parallel is the primary vehicle to achieve performance. By partitioning a layer of DNNs into multiple sub-layers, we can execute them in parallel on multicore NPUs. Within a core, we can also employ pipelined execution to reduce the execution time of a sub-layer. In this execution model, synchronizing parallel execution and loading/storing intermediate tensors in global memory are the main bottlenecks. To

alleviate these problems, we propose novel optimization techniques which carefully consider partitioning direction, execution order, synchronization, and global memory access. Using six popular convolutional neural networks (CNNs), we evaluate our optimization techniques in a flagship mobile SoC with three cores. Compared to the highest-performing partitioning approach, our techniques improve performance by 23%, achieving a speedup of 2.1x over single-core systems.

CCS Concepts: • Computer systems organization → Multi-core architectures; • Computing methodologies → Machine learning; • Human-centered computing → Mobile computing.

Keywords: Neural networks, Multicore, Neural processing unit, Layer partitioning, Scheduling optimization

ACM Reference Format:

Hanwoong Jung, Hexiang Ji, Alexey Pushchin, Maxim Ostapenko, Wenlong Niu, Ilya Palachev, Yutian Qu, Pavel Fedin, Yuri Gribov, Heewoo Nam, Dongguen Lim, Hyunjun Kim, Joonho Song, Seungwon Lee, and Hwansoo Han. 2023. Accelerating Deep Neural Networks on Mobile Multicore NPUs. In *Proceedings of the 21st ACM/IEEE International Symposium on Code Generation and Optimization (CGO '23), February 25 – March 1, 2023, Montréal, QC, Canada*. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3579990.3580015>

1 Introduction

Deep neural network (DNN) becomes a promising solution in modern smart devices, particularly for image processing applications that require classification, object detection and

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CGO '23, February 25 – March 1, 2023, Montréal, QC, Canada

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0101-6/23/02...\$15.00

<https://doi.org/10.1145/3579990.3580015>

scene segmentation. To increase the performance of such applications, many researchers have designed specialized accelerators to process DNNs efficiently [7, 9, 43]. Nowadays, most of the major smartphone manufacturers design their own accelerators, known as neural processing units (NPUs) and integrate them into their latest mobile SoCs [16, 32].

The performance of NPU continues to increase for the competitiveness of their products by accelerating DNN applications and AI benchmarks [12, 30]. Similar to the case of CPU, there is a traditional design issue for NPU: *a big single core versus small multiple cores*. Multicore NPUs typically bring many benefits, when concurrent execution of multiple DNNs as well as efficient processing of various sized networks is needed. In addition, to cope with the rapid product release cycle and market demand for various product lineups from *entry* to *flagship* at the same time, designing products with different number of NPU cores is a suitable approach for manufacturers.

Figure 1 shows a typical architecture of the latest mobile SoCs which include a multicore NPU. Multicore design consists of a compute engine and distributed, isolated local memory per core. Local memory is explicitly managed by programmers through load and store operations to global memory, and buffers around the compute engine are automatically managed by hardware logic.

A major hurdle for multicore designs in mobile NPUs is efficiently accelerating the inference of a single neural network. Since most DNN usages on mobile devices are inferences on a single image input, the latency of DNN inference is a key metric to evaluate performance. Parallelization is still a viable approach to enhancing performance, but we need to devise parallelization techniques for latency improvement. Recently, parallelization of DNN execution has been intensively investigated. Although many research works have focused on distributed DNN training on a large scale cluster of multi-GPU nodes [4, 26, 28, 34], techniques used in these works can be still effective to multicore NPU inference.

Parallelization approaches used in distributed DNN training can be categorized as data, operator (or model), and pipeline parallelisms. Combining them, as known as 3D parallelism [29, 48], enables cluster of accelerator nodes to process

large scale DNN models. As inference on mobile devices usually works on a single image input, data parallelism, which decomposes a large number of input data into small sets, will not suit well on multicore NPU inference. On the other hand, operator and pipeline parallelisms [13, 23, 39, 42, 48, 51] can be adopted to mobile multicore NPUs.

By partitioning an operator (a layer in DNN graph) to take multiple pieces of weights and/or input tensors, operator parallelism can be employed. Pipeline parallelism can also be employed within a core by splitting inputs into multiple tiles and executing them in a software pipeline. The main benefit of single-core tiling comes from overlapping global memory accesses with kernel computations. Pipelining in distributed DNN training often employs a large number accelerators to form a pipelined execution and segmented sub-graphs of DNN form stages for the pipeline. However, mobile devices usually have only a few number of cores, which is too few to form a long multi-stage pipeline. Moreover, considering that mobile NPU inference typically takes a single image input, we can easily conclude that pipelining across multiple NPU cores is inadequate on mobile devices.

In this paper, we adopt layer partitioning for operator parallelism across multiple NPU cores and input tiling for pipeline parallelism within a single core. While employing such parallelisms, we found many opportunities to eliminate the overheads associated with synchronization across multicores and communication between local memory and global memory. Optimization techniques make many decisions including execution order of layers, partitioning direction, partitioning size for NPU cores, trade-off between communication and redundant computation, and tile execution order. Our approach breaks down the original problem into two sub-problems: partitioning for parallelization across multicores and tiling for pipelining on a single core. Key contributions are summarized as follow:

- We define the optimization problem of parallelisms for multicore NPUs which considers the complex correlation between workload balancing, data reusability, synchronization, and computation redundancy. The problem has not been addressed by prior work.
- We propose execution models for multicore NPU systems: partitioning for multicore parallelization and tiling for single core pipelining, which reduces the intrinsic overheads of multicore NPU parallelisms.
- We propose optimization techniques to improve performance by eliminating synchronization and scheduling data exchange early.

The rest of this paper is organized as follows. Section 2 describes the overall execution models on multicore NPUs. Section 3 introduces details of proposed optimizations. Section 4 shows experimental results. Section 5 discusses related work, and we conclude our paper in Section 6.

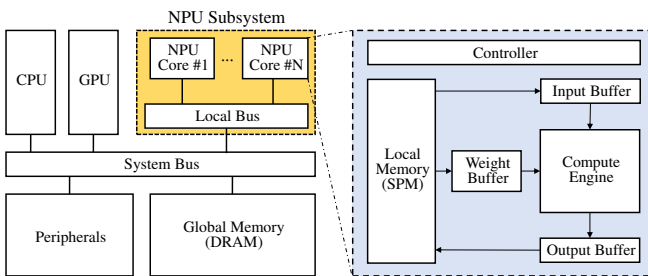


Figure 1. Architecture of mobile SoC with multicore NPU

2 Execution Models for Multicore NPUs

First, we describe a base execution model for DNN inference on a single core NPU system. Figure 2 shows the base execution model for a single neural network on a single core. In this base model, the NPU subsystem is equipped with a single compute engine along with local memory. Since NPUs on mobile devices are designed within a small area of the SoC, its local memory is just large enough to hold only the input/output tensors and the kernel weights of one layer of the neural network.

Figure 2 (a) shows a neural network containing five layers, $l_0, l_1, \dots, l_4$. To perform an inference on the neural network, we sort the layers in topological order and execute one layer by another in that order. To execute a layer (l_k) on NPU, we load its input tensor (*input* l_k) and kernel weights (*kernel* l_k) from global memory, compute its output tensor (*output* l_k), and store the output in global memory. As shown in Figure 2 (b), we use the local memory of NPU to store the input, output, and kernel for the currently executing layer. For each layer execution, we follow the three steps: ① *load* l_k , ② *compute* l_k , ③ *store* l_k . While executing layers, we use the global memory in the mobile SoC to store the intermediate output tensors from the previously executed layers, which will be loaded later as input tensors to the local memory for layer execution. In addition, all the kernel weights of the all layers are stored in the global memory. They will be loaded to the local memory, when its corresponding layer executes. The top layer takes the user input and the output computed from the bottom layer is the final output of the execution.

2.1 Parallel Execution of Partitions on Multicores

Figure 3 shows an example of layer partitioning, along with parallel execution of partitions on a three core NPU. To parallelize DNN execution, the original neural network is partitioned into sub-layers. Figure 3 (a) shows a neural network with three layers (l_0, l_1 , and l_2) and Figure 3 (b) shows how each layer is partitioned into sub-layers. Assuming three core NPU subsystem, layer l_0 is partitioned to three sub-layers

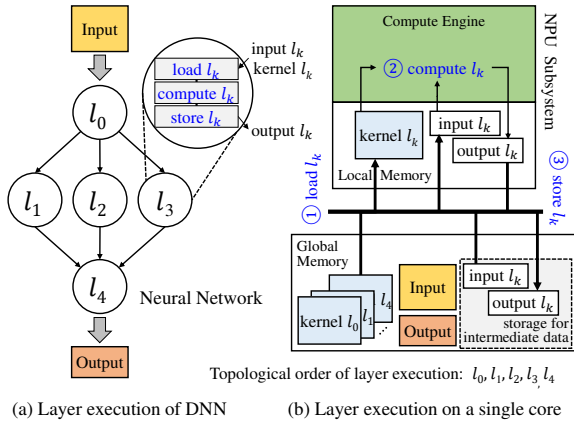


Figure 2. Layer execution of neural network for NPU

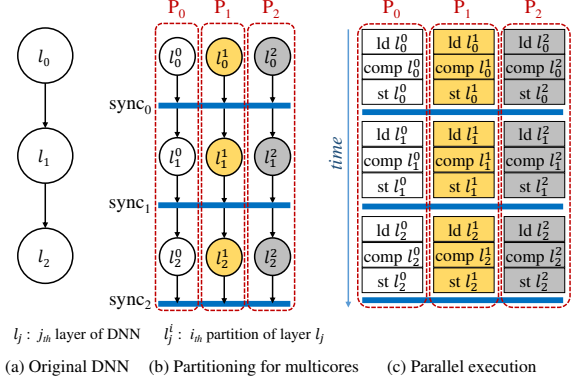


Figure 3. Parallel execution with partitioning (multicore)

(l_0^0, l_0^1 , and l_0^2). The superscripts of sub-layers indicate the processor cores to run on, respectively. The rest of the layers are also partitioned into three sub-layers. To maintain the same execution semantics, the sub-layers from a layer are synchronized at the end. The dotted lines indicate the boundaries of cores. Within a core, sub-layers are executed in the original topological order with synchronizations with other cores. Figure 3 (c) describes layer execution in detail. Each sub-layer l_j^i (i_{th} partition of the j_{th} layer) consists of *load* l_j^i , *compute* l_j^i , and *store* l_j^i , where it loads the corresponding input tensors and kernel weights for the sub-layer l_j^i , computes the output and stores it for that sub-layer. Operator partitioning [20, 42, 48] is well-investigated for parallelization. We adopt similar principles to parallelize them. On top of that, we will introduce several optimization techniques to minimize synchronization overhead by removing or hiding it and optimization problem between them.

2.2 Pipelined Execution of Tiles within a Core

Figure 4 shows an example of tiling for sub-layers. Within a single core (P_i), the partitions of the sub-layers are further divided into tiles, as shown in Figure 4 (b). The first sub-layer l_0^i is split into two tiles, $l_{0,0}^i$ and $l_{0,1}^i$. The second sub-layer l_1^i

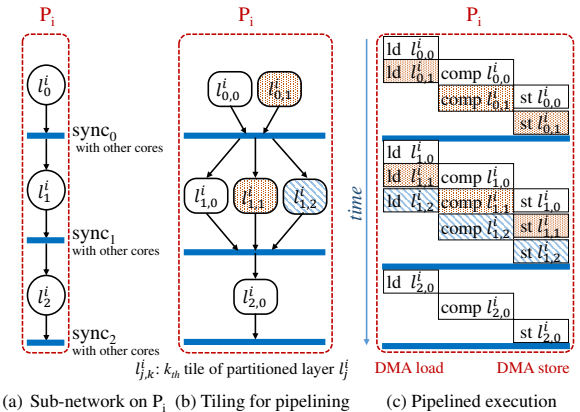


Figure 4. Pipelined execution with tiling (single core)

Table 1. Layer partitioning methods for convolution

Partition direction	Data partitioned	Extra replicated	comm. & comp.
spatial	input, output	kernel	none
spatial*	kernel	input, output	partial sum reduction
channel	kernel, output	input	none
channel*	input, kernel	none	partial sum reduction

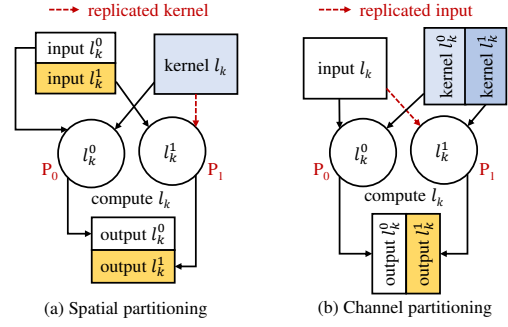
* Not preferred due to extra communication and computation across cores

is split into three tiles, $l_{1,0}^i$, $l_{1,1}^i$ and $l_{1,2}^i$. The third sub-layer $l_{2,0}^i$ constitutes one tile, $l_{2,0}^i$ as a whole. Tile sizes are determined to overlap global memory accesses with computations. Thus, double buffering on local memory minimizes the execution times of sub-layers in software pipeline of *load*, *compute*, and *store*. This is illustrated in Figure 4 (c). Double buffering combined with pipelining can speed up sub-layer execution on a single core, as the global memory access via DMA can overlap with computations. It also effectively reduces the space requirements of local memory. As only twice the size of each tile is required for input, weight, and output, local memory is saved when three or more tiles are used. The main reason for applying tiling within a single core is not just to improve data locality from reuse in our case. In addition, it is to hide the communication overhead to global memory with double buffering and pipelining, which is also extensively studied in stencil computation [37, 40]. We will enhance pipeline scheduling by having boundary data, called *halo*, ready early for the next layer computation.

3 Multicore Optimizations in Compiler

In the compilation phase, the layers of the given network are decomposed to sub-layers, which will be executed individually on a single core. Table 1 shows all possible layer partitioning methods of convolution layers as an example. A layer can be partitioned by splitting one of its input data, kernel weights, output data. Depending on layer operations, it is necessary to partition and/or replicate the rest of its data additionally. The compiler can be given multiple choices for partitioning directions. For an image data, spatial and channel directions are typical choices. The first and the third rows of Table 1 are typical methods for spatial and channel partitioning, respectively. The second and the forth rows, which are marked with asterisks (*), are alternative spatial and channel partitioning methods, but they require extra communication and computation for partial sum reduction. Compared to the first and third methods, these methods are undesirable in parallel execution, as we need to add extra stages for partial sum reduction.

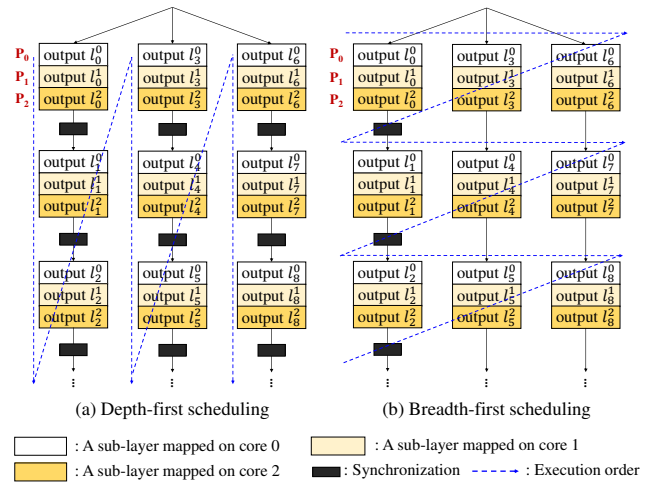
For the convolution layer partitioning methods specified in the first and third rows of Table 1, Figure 5 depicts two parallel execution schemes for the layers partitioned in spatial and channel directions, respectively. The diagrams are abstractions to show that a layer l_k is partitioned into two sub-layers, l_k^0 and l_k^1 , and illustrates how input tensors, output tensors and kernel weights are loaded and stored between

**Figure 5.** Layer partitioning for convolution

the global memory and the local memory of each processor (solid lines). They also indicate which data are replicated on the local memory of processing cores (dotted lines). In spatial direction partitioning (Figure 5 (a)), input and output tensors are partitioned in spatial directions, that means images are segmented in half along the height or width. Each partition will be computed on each of two processing cores in parallel. Since the convolution layer needs to apply all the kernel weights to compute the one element of the output tensor, kernel weights are replicated on both processors. Channel direction partitioning (Figure 5 (b)) produces an output image partitioned along channels. Since the convolution layer has a separate set of kernel weights for each channel, the kernel weights and the output tensors are segmented in half along the channel directions, but the input tensors are replicated in both local memories of both processing cores.

Before we propose our compiler optimizations, we need to discuss several aspects of partitioning and tiling techniques.

1) Data redundancy. Sub-layers produce their output tensors on multiple processing cores and those output tensors constitute the whole output tensors of the layer. The required input tensors and kernel weights are calculated to perform computations in parallel. Depending on operations, some of the data need to be fed to multiple processing cores. For a convolution layer partitioned in spatial direction as an

**Figure 6.** Layer scheduling strategies

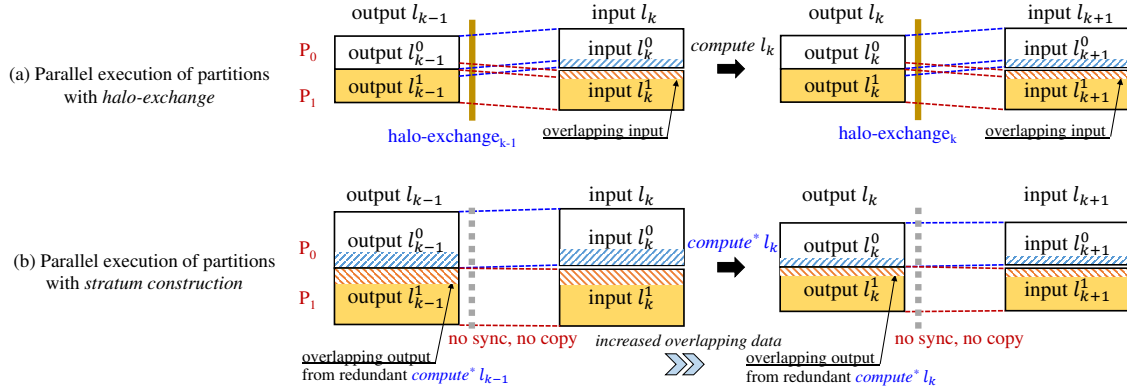


Figure 7. Data forwarding with *halo-exchange* and *stratum* construction

example, the whole kernel weights are duplicated on multiple processing cores. In addition, borderline data, called *halo*, should be duplicated on input partitions. Since convolution masks (kernel weights) slides over the input image and produce effects of blurring, sharpening, embossing, edge detection, etc, the data near partition boundary should be needed on multiple processing cores.

2) Data dependency. Deep neural network forms a graph with directed edges. The nodes representing layers should be executed in a certain order (e.g., topological order) to satisfy the enforced dependencies by the directed edges. Before we begin the computation of a layer, the output tensors from its preceding layers should be all ready. Thus, we have to set up an appropriate execution order of layers. Maintaining topological order, we can resort to depth-first or breadth-first scheduling strategy as shown in Figure 6.

3) Data reusability. Output tensors of a layer eventually become input tensors for its immediately succeeding layers in a directed graph. When a succeeding layer is scheduled right away at the same processing core, output tensors can be forwarded as input tensors within local memory without having to store and load them from global memory. This optimization, called *feature-map forwarding* in image processing terms, provides great benefits in terms of performance, power, and memory bandwidth. When local memory is large enough to hold the output tensors for a while, we can have a chance to forward them to multiple immediate succeeding layers by executing those layers. Even though we are not forwarding them right away, we might keep the output in local memory and find chances to forward them later.

4) Data exchange. For consecutively running layers, some of output tensors of preceding layers are likely to be reused in the succeeding layers. Instead of accessing global memory to deliver data between layers, direct data exchange among processing cores will save the memory bus bandwidth. The borderline data (*halo*) is a great example, which is shown in Figure 7 (a). Output tensors usually contain no redundant data, but the layers who need them as inputs additionally want *halo* data from neighboring cores. Thus, *halo* data are

exchanged and replicated among multiple cores by using a special interface called *halo-exchange*.

5) Computation redundancy. When a layer is partitioned and executed in parallel on multiple cores, synchronization is required at the end of layer execution. Since our parallel execution model stores outputs in global memory and loads them as inputs for the later layer executions. Thus, synchronization ensures that all the data are written to global memory before being read. Synchronizations often cause performance degradation in multicore execution environments. Eliminating them, even at the cost of redundant computation, improves performance. Accumulating layers that can be executed without synchronizations and global memory accesses (copies) is called *stratum* construction. Some layer operations as shown in Figure 7 (b) require redundant data (for input, output, and kernel) and computation (for overlapping data) to remove synchronizations. For multiple consecutive convolution layers, the amount of redundancies in data and computation increases toward the top layers in a *stratum*, but once you build a *stratum*, all multicores can execute them independently within a single core until the last layer.

These aspects of optimizations mentioned above are intertwined with each other. For example, scheduling layers in depth-first order, as shown in Figure 6 (a), increases chances of data reuse. This allows us to apply *feature-map forwarding* along with *halo-exchange* and *stratum* construction. On the other hand, scheduling layers in breadth-first order, as shown in Figure 6 (b), eliminates some of the data dependencies among immediately following layers. This allows

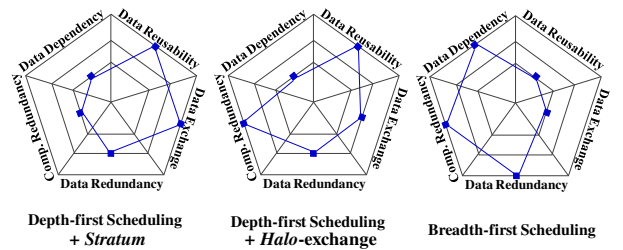


Figure 8. Comparison of optimization strategies

the span between synchronization points to be extended, which in turn can reduce synchronization overhead. However, breadth-first order scheduling will reduce the chance of data reuse between consecutively executed layers. Figure 8 compares several optimization strategies in terms of the optimization aspects discussed above. Each optimization strategy has different preferences for each aspect. That is, optimization problem can not be solved only by the single optimization strategy, but solved by a mixture of different strategies for each part of the neural network.

3.1 Optimizations for Parallelization and Pipelining

In this section, we introduce how the NPU compiler schedules layer executions along with layer partitioning heuristics for parallelization. Then, we describe *stratum construction* to further optimize schedule by eliminating synchronizations. Finally, we improve the pipelined schedule within a single core to help start *halo-exchange* as early as possible.

3.1.1 Layer Scheduler. Partitioning direction affects the execution order, which in turn impacts on *data exchange* and *data reusability* as well as *data dependency* between adjacently scheduled layers. When partitioning layers, we prefer a certain direction depending on data and operation characteristics. For successive convolution layers, partitioning in spatial direction is preferred, as it is suitable to exploit temporal data locality between them. In theory, partitioning direction is actually hard to determine, as it is affected by all optimization aspects addressed in the previous section. Thus, we determine partitioning direction by using following heuristics. Spatial and channel directions in the following heuristics are applicable only to image data, but heuristics can be generalized with the same concerns on data reusability, data shape, operation type, and data exchange.

- *h1 (default)*. Spatial partitioning is a default option due to better data reusability than channel partitioning, but channel partitioning is used in the cases of *h2-h5*.
- *h2 (data reuse)*. While spatial partitioning requires kernel weights to be replicated, channel partitioning needs to replicate input tensors. If kernel weights are too large, compared to input tensors, channel partitioning is preferred to reduce redundant data copies on local memories.
- *h3 (data shape)*. If the shape of input is too shallow to be split in spatial direction over all cores, channel partitioning is preferred to increase parallelism.
- *h4 (operation type)*. For channel-wise operations (e.g., depth-wise convolution and pooling), channel partitioning is preferred, as it does not require redundant data copy for kernel.
- *h5 (data exchange)*. If spatial partition incurs too large *halo* data to exchange due to operation characteristics (e.g., kernel, stride, and dilation rate of convolution), channel partitioning could reduce the amount of data to exchange.

Algorithm 1: Layer execution schedule

```

1 Input : A neural network  $N$ , a set of top layers  $in(N)$ ;
2 Output: An ordered set of layers  $S_{out}$ ;
3  $S_{ready} \leftarrow in(N)$ ;  $l_{cur} \leftarrow S_{ready}.getfirst()$ ;
4 while  $S_{ready} \neq \emptyset$  do
5    $S_{out} \leftarrow S_{out} \cup \{l_{cur}\}$ ; // schedule current layer
6    $S_{ready} \leftarrow S_{ready} - \{l_{cur}\}$ ;
7    $S_{ready} \leftarrow S_{ready} \cup succ(l_{cur})$ ;
8   // get a successor in depth-first traversal tree
9    $l_{succ} \leftarrow S_{ready}.get\_succ(l_{cur})$ ;
10  // get current or ancestor's sibling in depth-first traversal tree
11   $l_{sibling} \leftarrow S_{ready}.get\_sibling(l_{cur})$ ;
12  if both  $l_{succ}$  and  $l_{sibling}$  are available then
13    if  $spatial\_partitioning(l_{cur})$  then  $l_{next} \leftarrow l_{succ}$ ;
14    else  $l_{next} \leftarrow l_{sibling}$ ;
15  else
16     $l_{next} \leftarrow either\_of\_available(l_{succ}, l_{sibling})$ ;
17   $l_{cur} \leftarrow l_{next}$ ;

```

Algorithm 1 shows how we determine an execution order of layers. It starts from one of input layers of the given network, and tries to choose two candidate layers ready to be executed from S_{ready} : one layer (l_{succ}) is dependent on the current layer (l_{cur}) — l_{succ} is a successor of the current layer, and the other ($l_{sibling}$) is a sibling or ancestor's sibling layer in depth-first traversal tree — $l_{sibling}$ has no data dependencies on the current layer. Then, *spatial_partitioning()* in line 15 of Algorithm 1 determines the next layer to schedule. It is implemented according to the heuristics we described for partition directions (*h1-h5*). General idea of schedule order is that you choose the successor layer if you can increase performance by applying data reuse with the currently scheduled layer; otherwise, you choose the sibling layer, as there is little benefit in data reuse from spatial partitioning.

Once the partitioning direction is determined, we determine the partitioning size for each core. Multicores in the NPU subsystem can be heterogeneous in computing power and memory bandwidth. To achieve performance without wasting cycles in the parallel execution of partitioned layers, the workload should be balanced among the cores. Thus, the partitioning ratio of each core is determined according to its computing power and memory bandwidth to global memory. As a result, the total time of accessing memory and executing kernel should be well-balanced across cores. In addition, memory accesses for data may have alignment constraints. When calculating the amount of data to load and store in global memory, we should also consider alignment constraints of NPU core for better load balancing.

3.1.2 Optimization with Stratum Construction. We also perform *stratum* optimization for further improvement by eliminating synchronizations. Constructing a *stratum* of layers starts by visiting the scheduled layers in reverse order. As shown in Figure 7 (b), we need to include borderline data for the next layer in the output redundantly, which

allows us to eliminate synchronizations. In turn, we need to compute redundantly to produce borderline (*halo*) data in the output. For this redundant computation, we may also need to have more redundant input than output. In this way, *stratum* construction starts from the layer in the bottom and builds an accumulation of layers backward in neural network. The more redundant data is required toward the higher layer in the accumulation (*stratum*). Stratum construction proceeds based on the following heuristics:

- *h6 (immediate successor)*. If consecutively scheduled layers are directly connected to each other as successor and predecessor layers, they can be candidates to construct a *stratum*. Since the main benefit of *stratum* comes from *feature-map forwarding* and synchronization elimination, they should be scheduled adjacent to each other.
- *h7 (partitioning directions match)*. To make redundant data and computation minimum, two adjacent layers in *stratum* should have matching partitioning directions. In convolution partitioning, they should be both spatial. Since consecutive channel partitioning still needs to have all the input tensor in every layer, layers with channel partitioning does not match for *stratum* construction.
- *h8 (redundant computation is cheap)*. The estimated cost of redundant computation should be cheaper than expected cost of synchronization. In *stratum* execution, no data copies to and from global memory. With no synchronization, each core computes locally owned data across multiple layers. Only overheads are redundant computations in those layers. Since we replace synchronizations with redundant computations, the cost comparison in each layer to accumulate should justify the *stratum* construction. Otherwise, *stratum* construction should stop and fresh start from that layer for *stratum* construction.

Based on the above heuristics (*h6-h8*), Algorithm 2 shows how we construct *stratum* chains in reverse direction of layer scheduling. The **if** conditions in lines 6–8 check the requirements addressed in the heuristics one by one. Then, we determine to include the current layer into the current stratum (*cur_stratum*) and also recalculate the output size of the current layer to include the redundant *halo* data for the next scheduled layer. With this adjustment, we correctly reflect the increase to the input size and we will propagate toward the upper layer in *stratum* chain. When the current layer does not bring benefits, the algorithm stops the layer accumulation to the current *stratum* and adds the layers accumulated so far to *strata_set* as a new *stratum*. Then, it resets the accumulation and fresh starts *stratum* accumulation by setting the current layer as a new base layer.

The cost calculations in *redundant_compute_cost()* and *sync_cost()* need machine specific data to estimate them accurately. Thus, we measure the necessary operator cost per element on our NPU hardware and also acquire the profile data for synchronization per layer. Based on those measured data,

Algorithm 2: *Stratum* construction

```

1 Input : Scheduled layers  $\{l_j\}$ ; ordered in execution schedule;
2 Output: Set of constructed strata strata_set;
3 // start stratum construction with the last scheduled layer as base
4 strata_set  $\leftarrow \emptyset$ ; cur_stratum  $\leftarrow \{l_{last}\}$ ; l_prev  $\leftarrow l_{last}$ ;
5 for  $l_{cur} \in \{l_j\}$ : all layers in a reverse scheduled order -  $\{l_{last}\}$  do
6   if  $l_{prev} == succ(l_{cur})$  and // immediate successor in DNN
7     partition_direction_match( $l_{cur}, l_{prev}$ ) and
8     redundant_compute_cost( $l_{cur}$ ) < sync_cost( $l_{cur}$ ) then
9     // accumulate current layer to current stratum
10    if  $|cur\_stratum| > 0$  then
11      // increase output to include halo for successor layer
12      for  $i \in \text{all multicores}$  do
13        output_size( $l_{cur}^i$ )  $\leftarrow output\_size(l_{cur}^i) +$ 
14          halo( $succ(l_{cur}^i)$ )
15    cur_stratum  $\leftarrow cur\_stratum \cup l_{cur}$ ; l_prev  $\leftarrow l_{cur}$ ;
16  else
17    // stop accumulating layer, save current stratum
18    if  $|cur\_stratum| > 1$  then
19      strata_set  $\leftarrow strata\_set \cup cur\_stratum$ ;
20    // fresh start by retrying current layer as a new base
21    cur_stratum  $\leftarrow l_{cur}$ ; l_prev  $\leftarrow l_{cur}$ ;

```

we empirically implement the two cost estimation functions. Another clarification to note is that *stratum* construction also increases the required memory size to execute layers. However, it is not considered at this stage. Later, pipelining with tiling will have a chance to reduce the required local memory. If this cannot be resolved at the final stage of compilation, the compiler will eliminate layers from the top of the *stratum* until it fits on actual local memory to run on.

3.1.3 Optimizations with Tiling. Layers are partitioned into sub-layers that are units of parallel execution on multicores. A sub-layer is decomposed again within a single core, if 1) the amount of required memory is larger than the local memory or 2) the data transfer time can be hidden by overlapping the computation. The fragments decomposed from a sub-layer are called *tiles*. Previous work [20, 35, 47] showed that the main optimization criterion for tiling is to exploit data locality in memory access to maximize data reusability. On the other hand, in our case, the main role of the tiling is to reduce execution time by software pipelining. Three stages of *load*, *compute*, and *store* for a tile ($l_{j,k}^i$, k -th tile of sub-layer l_j^i) are pipelined, overlapping the computation with memory accesses within a single core (P_i). The memory requirement for a tile is reduced by the number of tiles and pipelining is often implemented with double buffers for the tile data. Thus, if we have three and more tiles, we can reduce the amount of required memory in addition to performance benefits.

We consider all possible combinations of tiling directions to meet the local memory size and alignment requirements. When the sub-layers to be tiled are spatially partitioned, tiling them in the same direction can hide *halo* data transfer overhead. When they are partitioned in any other direction,

the tiling direction is determined to maximize data locality, similar to existing works [20, 35]. One caveat is that the NPU compiler compiles sub-layers independently and may produce contradicting tile directions across multiple cores. This may incur unbalanced workload across multicores and unnecessary idle time as a consequence. Thus, profiling execution assists to detect unwanted idle times and fix the unbalance. For tile execution order within a single core, we first schedule the tiles that produce *halo* data to the next layer if the layer is partitioned spatially (*halo-first policy*). If it is partitioned by more than two directions including spatial direction, the tiles contributing to *halo* data are scheduled in a data locality maximizing order to minimize the data transfer between local memory and global memory.

3.2 Halo vs. Stratum

When two directly connected layers are scheduled consecutively, a large portion of output tensors of the preceding layer may be passed to the succeeding layer as input tensors. The rest of input tensors can be obtained from the other processing cores that compute the adjacent partitions. Those borderline data can be exchanged among cores with *halo-exchange*. Figure 9 (a) shows the parallel execution of two layers (l_1 and l_2). At the end of the first layer execution, all processing cores invoke *halo-exchange* to get the borderline data for the second layer execution. Instead of a series of store, sync, and load ($st\ l_1 - sync_1 - ld\ l_2$) for data passing to the second layer, *halo-exchange* followed by load-kernel of the second layer (*halo-exch - ld-kn l_2*) is enough to ready the second layer computation. Usually, *halo-exchange* will reduce the amount of data traffic on the bus and the elapsed time as well in parallel execution.

Moreover, if we apply pipelining within a single core as shown in Figure 9 (b), the execution time will be saved greatly again. We only split the first sub-layer into three tiles and the second sub-layer constitutes one tile as a whole, as it is enough to show the benefit of pipelining. With *halo-first policy*, the tiles producing the *halo* data will be scheduled first in pipeline (we assume that $l_{1,2}^i$ calculates all *halo* data in the diagram). If *halo-exchange* totally overlaps with the computations for the rest of the tiles ($l_{1,0}^i$ and $l_{1,1}^i$), we will have no

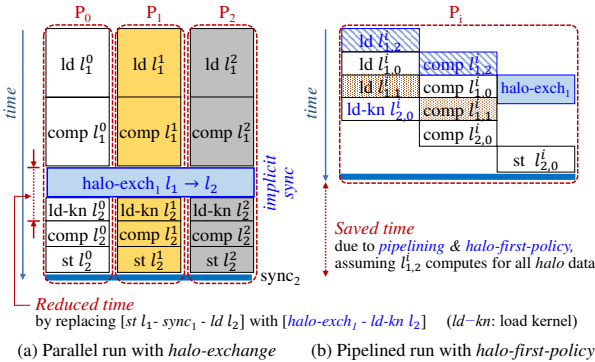


Figure 9. Time reduction from *halo-exchange*

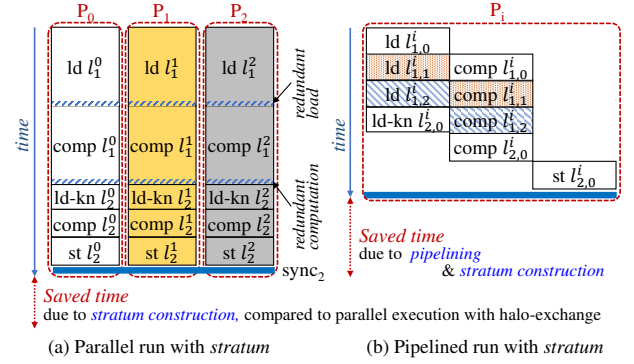


Figure 10. Time reduction from *stratum construction*

exposed overhead for borderline data communication. One additional benefit for the second layer is that kernel load for the second layer ($ld-kn\ l_{2,0}^i$) can start early, while the last tile of the first layer is computing ($comp\ l_{1,1}^i$).

Now, assume a *stratum* consists of two consecutively scheduled layers (l_1 and l_2). Layers in a *stratum* are executed with no synchronizations and no data passing across cores. Thus, all cores compute the first layer (l_1) after loading its input tensors and kernel weights. Then, its output tensors are just the input tensors for the second layer (l_2). From the second layer, the layers only need to load kernels (*ld-kn*). Thus, *stratum construction* saves the execution time by eliminating synchronizations, output stores, and input loads, which is depicted in Figure 10 (a). However, when a layer is added to a *stratum*, it will add redundant computations and redundant data to load, which is presented as hatched portion of load and compute in the diagram. Thus, the compiler needs to stop accumulating layers, when its redundancy outweighs the synchronization and memory accesses. We can also apply pipelining along with *stratum construction*. Figure 10 (b) shows a pipelined execution, when we split the first sub-layer of a core (P_i) into three tiles and take the second sub-layer as one whole tile — this is the same as the *halo* pipelining. Since we have only load (or load kernel) and compute for tiles of sub-layers, pipelining can cover all tiles of the sub-layers in a *stratum* straightforwardly.

Halo optimization can hide the overhead of *halo-exchange* by overlapping with computations for other tiles. Since *stratum* optimization requires redundant computations and data load, *halo* optimization may look better than *stratum* optimization. However, *halo-exchange* may hinder the current load stages for other tiles, as it still takes up the bandwidth of the system bus. Moreover, it still implicitly imposes synchronization and may take a heavy toll depending on dynamic situations of the system. On the other hand, *stratum construction* eliminates synchronizations and data exchanges with the other cores. Compared to the main data to process, its borderline data and redundant computations are very small in practice. Thus, *stratum* optimization often results in better performance than *halo* optimization.

Table 2. Benchmark CNN models

Category	Model	Input size (H×W×C)	Data type
Classification	InceptionV3 [36]	299 × 299 × 3	INT8
	MobileNetV2 [33]	224 × 224 × 3	INT8
Object detection	MobileNetV2-SSD [22]	300 × 300 × 3	INT8
	MobileDet-SSD [41]	320 × 320 × 3	INT8
Segmentation	DeepLabV3+ [6]	513 × 513 × 3	INT16
	UNet [31]	572 × 572 × 3	INT8

4 Experiments

We evaluated the proposed optimization methods on Samsung Exynos 2100 [32] SoC which has three NPU cores [17, 27]. Each core consists of an adder-tree-based inner-product engine, so data alignments for input and output channels are fixed for each. Also they have different bandwidth capabilities. All these factors are taken into account in the compilation phase. For benchmark, we use well-known CNNs listed in Table 2. To show the effectiveness of our optimizations, we performed evaluation runs on several configurations as listed in Table 3. Our baseline scheme, BASE, chooses partitioning direction of each layer and determines layer execution schedule as described in heuristics (*h1-h5*). It uses a similar scheme to SmartShuttle [20], which proposed an adaptive layer tiling according to data shape and off-chip memory access volume. Cumulative optimizations are applied in *+Halo* and *+Stratum*. In *+Halo*, *halo-exchange* reduces data traffic and *halo-first policy* reduces the overhead by overlapping with computations for other tiles. In *+Stratum*, *strata* are constructed to eliminate synchronization and global memory access at the cost of increased computation.

4.1 Experimental Result

In Figure 11, we show the performance comparison for benchmark models. Understandably, performance is improved by reducing data exchange (*Halo*) and eliminating synchronization (*Stratum*). We also find several notable points from the results. First, the performance is considerably good on a single core. Thus, BASE configuration with three cores barely achieves 2x speedups over single core runs. Since inference times for all models are only a few milliseconds, even a small amount of inefficiency makes its overhead look large. Second, optimizations in *Halo* and *Stratum* sometimes degrade performance. When we additionally optimize with *Stratum*, InceptionV3 shows slightly lower performance than *Halo*.

Table 3. Configuration for cumulative optimizations

Configuration	Description
BASE*	Parallelized with partitioning layers and pipelined with tiling sub-layers
<i>+Halo</i>	Cumulatively, optimized with <i>halo-exchange</i> and pipeline scheduled with <i>halo-first policy</i>
<i>+Stratum</i>	Cumulatively, optimized with <i>stratum construction</i> for parallel, pipelined run

DeepLabV3+ performs worse with *Halo* than BASE. Since the heuristics for our optimizations rely on profiling data, the dynamic nature can adversely affect optimization results.

Overall, *Halo* and *Stratum* achieve further improvement cumulatively from BASE by 1.07× and 1.23×, respectively. On average, it is 2.1x faster than single-core execution. Given these results, we confirm that the optimization techniques addressed in this paper effectively improve the performance of neural network inference on multiple NPU cores. Detailed analysis on experimental results are shown below.

1) Effects of partitioning schemes. As BASE uses *adaptive partitioning*, all configurations are configured to use adaptive ones. To check the effects of partitioning schemes, we compare *adaptive partitioning* to both *spatial partitioning* and *channel partitioning* in terms of the total data transfer amount and idle time on each core. By comparing the total amount of data transfer, we can check how much data transfer is increased due to *data redundancy*. In addition, the partitioning direction affects workload balancing due to the alignment constraints — channel direction imposes a relatively larger alignment requirement than the spatial direction. If the number of channels is too small to distribute evenly across the cores, core utilization can be degraded.

Table 4 compares the amount of data transfer and idle time on each NPU core for InceptionV3. *Adaptive partitioning* has the smallest amount of total data transfer and the least idle time on average (μ) across the three cores. It also achieves the lowest standard deviation (σ) among cores. It confirms that partitioning in one direction across all layers is unsuitable for maximum core utilization. It can cause inefficient use of system bandwidth due to redundant data transfer between global and local memories while processing cores run idle.

2) Effects of halo-first policy in pipelining. Figure 12 shows profile data of the first two convolutional layers in InceptionV3. The top three lines represent two *loads* of input and weight, and *store* of output, respectively for Core0. Then,

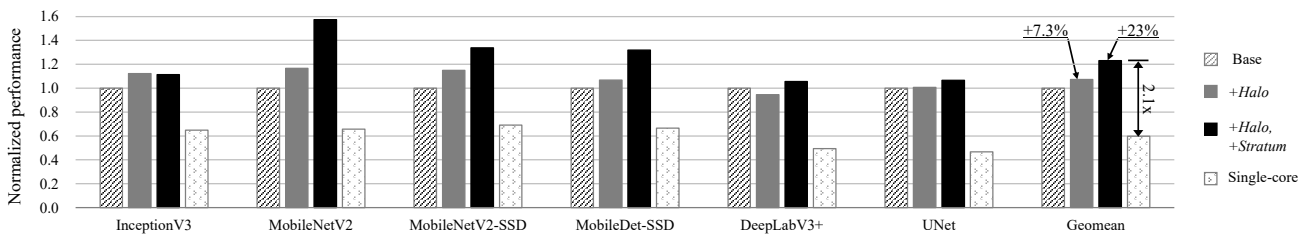
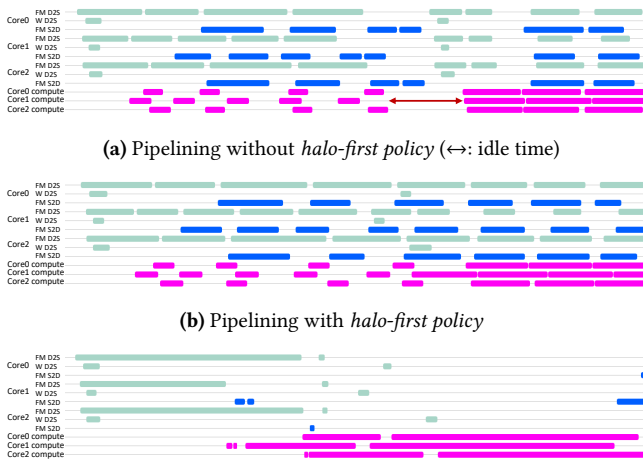
**Figure 11.** Performance comparison of benchmark models (performance = $\frac{1}{latency}$)

Table 4. Detailed profile data of InceptionV3

Partitioning scheme		Data transfer amount (global ↔ local mem.)	Idle time
Spatial partitioning	P0	31,397KB	145 us
	P1	22,230KB	1,090 us
	P2	17,708KB	1,077 us
Channel partitioning	P0	32,151KB	163 us
	P1	36,427KB	607 us
	P2	32,573KB	147 us
Adaptive partitioning*	P0	22,177KB	206 us
	P1	25,462KB	427 us
	P2	22,224KB	199 us

the following six lines of global memory accesses are depicted for *Core1* and *Core2*. The bottom three lines represent *compute* each for three cores. First, we compare two different cases: pipelining (a) without and (b) with the *halo-first policy*. Figure 12 (a) shows idle time, indicated with an arrow, to wait for *halo* data transfer completion. In Figure 12 (b), the next layer proceeds to compute immediately after the completion of previous layer. Figure 12 (c) shows the profiled schedule when we employ both *halo-first policy* and *feature-map forwarding*. There is no data load from global memory except *halo* data exchange. Moreover, its transfer is overlapped with computation. The results confirm that the *halo-first policy* effectively hides data transfer overhead, even when transferring *halo* data through global memory, due to no direct connection or shared memory between cores.

3) Effects of stratum optimization. To show the effectiveness of *stratum* construction, compared to *halo* exchange optimization, we perform three tests with different configurations: *+Halo* only, *+Stratum* only, and both combined. Table 5 shows the result of *stem* region in InceptionV3. Applying *halo* exchange to all layers does not increase the computational amount, but introduces a large amount of synchronization overhead. On the other hand, *stratum* has less synchronization overhead than *halo* exchange, but increases the amount of computation due to overlapping areas. As a result, the



(c) Pipelining with *halo-first policy* and *feature-map forwarding*
Figure 12. Pipelining profile for *halo-first policy*

Table 5. Comparison of *Halo* and *Stratum*

Configuration	End-to-end latency	Computation amount	Synchronization overhead
<i>+Halo</i>	387 us	1.34G	μ :21.2 us, σ :9.1 us
<i>+Stratum</i>	386 us	1.39G	μ :17.5 us, σ :9.2 us
<i>Combined</i>	378.8 us	1.35G	μ :14.2 us, σ :7.5 us

* A *stem* region of InceptionV3 is used.

best performance is achieved when the two optimizations are properly combined. Note that *feature-map forwarding* optimization can be applied to both optimizations if the size of local buffer is sufficient. In fact, *halo* exchange can have more chances of *feature-map forwarding*, as *stratum* comes with increased memory requirements.

5 Related Work

Recently, numerous researches have made efforts to optimize DNN inference for embedded systems. [2, 10, 14, 20, 21, 38, 49] propose neural network optimization techniques including layer tiling. [11] addresses how operator scheduling impacts on GPU performance. However, they do not consider multi-core partitioning and *inter-layer* optimization.

Also, [3, 44, 46, 50] propose a layer fusing (*stratum*) technique for a distributed computing environment, and [45] introduces a retraining technique, fully decomposable spatial partition (FDSP), to overcome a drawback of spatial partitioning: the borderline data should be shared across devices.

[19, 24] propose partitioning techniques to accelerate a single neural network on distributed devices. [18] proposes a scheduling technique based on Genetic Algorithm (GA) to run deep learning applications on heterogeneous processors. However, they do not consider fine-grained partitioning and scheduling problem for multi-NPU embedded systems.

Tensor Virtual Machine (TVM) [5, 8] supports various optimization techniques for DNNs such as loop tiling and unrolling based on Halide [15]. [25] introduces automated optimization heuristics, and a learning-based optimization technique is introduced in [1]. To our best knowledge, they do not consider *inter-core/layer* optimization for interoperation of tightly coupled NPU cores addressed in this paper.

6 Conclusion

In this paper, we propose parallelization and pipelining models for neural networks on mobile multicore NPU systems. We introduce key optimization criteria — *data redundancy*, *data dependency*, *data reusability*, *data exchange* and *computation redundancy*, and relevant optimizations such as *stratum* construction, *halo* exchange, and *halo-first policy*. Using multiple CNN models as benchmarks, we evaluate our optimizations and show a 23% performance increase over a sophisticated layer partitioning and scheduling scheme.

Acknowledgements

We would like to thank Ke Lu, Viacheslav Garbuzov, and all the developers who worked together on the NPU project.

References

- [1] Andrew Adams, Karima Ma, Luke Anderson, Riyadh Baghdadi, Tzu-Mao Li, Michaël Gharbi, Benoit Steiner, Steven Johnson, Kayvon Fatahalian, Frédo Durand, and Jonathan Ragan-Kelley. 2019. Learning to Optimize Halide with Tree Search and Random Programs. 38, 4, Article 121 (July 2019), 12 pages. <https://doi.org/10.1145/3306346.3322967>
- [2] Byung Hoon Ahn, Prannoy Pilligundla, and Hadi Esmaeilzadeh. 2019. Reinforcement Learning and Adaptive Sampling for Optimized DNN Compilation. <https://doi.org/10.48550/ARXIV.1905.12799>
- [3] M. Alwani, H. Chen, M. Ferdman, and P. Milder. 2016. Fused-layer CNN accelerators. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1–12.
- [4] Reza Yazdani Aminabadi, Samyam Rajbhandari, Minjia Zhang, Amar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Jeff Rasley, Shaden Smith, Olatunji Ruwase, and Yuxiong He. 2022. DeepSpeed Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale. <https://doi.org/10.48550/ARXIV.2207.00032>
- [5] Apache. 2020. TVM. <https://tvm.apache.org/>
- [6] Liang-Chieh Chen, Yukun Zhu, George Papandreou, Florian Schroff, and Hartwig Adam. 2018. Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation. In *ECCV*.
- [7] Tianshi Chen, Zidong Du, Ninghui Sun, Jia Wang, Chengyong Wu, Yunji Chen, and Olivier Temam. 2014. DianNao: A Small-Footprint High-Throughput Accelerator for Ubiquitous Machine-Learning. In *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems (Salt Lake City, Utah, USA) (ASPLOS '14)*. Association for Computing Machinery, New York, NY, USA, 269–284. <https://doi.org/10.1145/2541940.2541967>
- [8] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, Carlsbad, CA, 578–594. <https://www.usenix.org/conference/osdi18/presentation/chen>
- [9] Yu-Hsin Chen, Joel Emer, and Vivienne Sze. 2016. Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks. In *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*. 367–379. <https://doi.org/10.1109/ISCA.2016.40>
- [10] Antonio Cipolletta and Andrea Calimera. 2021. Dataflow Restructuring for Active Memory Reduction in Deep Neural Networks. In *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 114–119. <https://doi.org/10.23919/DATE51398.2021.9473965>
- [11] Yaoyao Ding, Ligeng Zhu, Zhihao Jia, Gennady Pekhimenko, and Song Han. 2021. Ios: Inter-operator scheduler for cnn acceleration. *Proceedings of Machine Learning and Systems* 3 (2021).
- [12] ETH. 2020. AI-Benchmark. <http://ai-benchmark.com/>
- [13] Shiqing Fan, Yi Rong, Chen Meng, Zongyan Cao, Siyu Wang, Zhen Zheng, Chuan Wu, Guoping Long, Jun Yang, Lixue Xia, Lansong Diao, Xiaoyong Liu, and Wei Lin. 2021. DAPPLE: A Pipelined Data Parallel Approach for Training Large Models. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '21)* (Virtual Event, Republic of Korea) (PPoPP '21). Association for Computing Machinery, New York, NY, USA, 431–445. <https://doi.org/10.1145/3437801.3441593>
- [14] Mingyu Gao, Xuan Yang, Jing Pu, Mark Horowitz, and Christos Kozyrakis. 2019. TANGRAM: Optimized Coarse-Grained Dataflow for Scalable NN Accelerators. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (Providence, RI, USA) (ASPLOS '19)*. Association for Computing Machinery, New York, NY, USA, 807–820. <https://doi.org/10.1145/3297858.3304014>
- [15] Halide. 2012. A language for fast, portable computation on images and tensors. <https://halide-lang.org/>
- [16] Huawei. 2019. Huawei launches Ascend 910, the world's most powerful AI processor, and MindSpore, an all-scenario AI computing framework. <https://www.huawei.com/en/news/2019/8/huawei-ascend-910-most-powerful-ai-processor>
- [17] Jun-Woo Jang, Sehwan Lee, Dongyoung Kim, Hyunsun Park, Ali Shafiee Ardestani, Yeongjae Choi, Channoh Kim, Yoojin Kim, Hyeongseok Yu, Hamzah Abdel-Aziz, Jun-Seok Park, Heonsoo Lee, Dongwoo Lee, Myeong Woo Kim, Hanwoong Jung, Heewoo Nam, Dongguen Lim, Seungwon Lee, Joon-Ho Song, Suknam Kwon, Joseph Hassoun, SukHwan Lim, and Changkyu Choi. 2021. Sparsity-Aware and Re-configurable NPU Architecture for Samsung Flagship Mobile SoC. *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)* (2021), 15–28.
- [18] Duseok Kang, Jinwoo Oh, Jongwoo Choi, Youngmin Yi, and Soonhoi Ha. 2020. Scheduling of Deep Learning Applications onto Heterogeneous Processors in an Embedded Device. *IEEE Access* 8 (2020), 43980–43991. <https://doi.org/10.1109/ACCESS.2020.2977496>
- [19] En Li, Liekang Zeng, Zhi Zhou, and Xu Chen. 2020. Edge AI: On-Demand Accelerating Deep Neural Network Inference via Edge Computing. *IEEE Transactions on Wireless Communications* 19, 1 (2020), 447–457. <https://doi.org/10.1109/TWC.2019.2946140>
- [20] Jiajun Li, Guihai Yan, Wenyan Lu, Shuhao Jiang, Shijun Gong, Jingya Wu, and Xiaowei Li. 2018. SmartShuttle: Optimizing off-chip memory accesses for deep learning accelerators. In *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 343–348. <https://doi.org/10.23919/DATE.2018.8342033>
- [21] Rui Li, Yufan Xu, Aravind Sukumaran-Rajam, Atanas Rountev, and P. Sadayappan. 2021. Analytical Characterization and Design Space Exploration for Optimization of CNNs. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (Virtual, USA) (ASPLOS 2021)*. Association for Computing Machinery, New York, NY, USA, 928–942. <https://doi.org/10.1145/3445814.3446759>
- [22] Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu, and Alexander C. Berg. 2016. SSD: Single Shot MultiBox Detector. In *ECCV*.
- [23] Lingxiao Ma, Zhiqiang Xie, Zhi Yang, Jilong Xue, Youshan Miao, Wei Cui, Wenxiang Hu, Fan Yang, Lintao Zhang, and Lidong Zhou. 2020. Rammer: Enabling Holistic Deep Learning Compiler Optimizations with rTasks. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 881–897. <https://www.usenix.org/conference/osdi20/presentation/ma>
- [24] J. Mao, X. Chen, K. W. Nixon, C. Krieger, and Y. Chen. 2017. MoDNN: Local distributed mobile computing system for Deep Neural Network. In *Design, Automation Test in Europe Conference Exhibition (DATE)*, 2017. 1396–1401.
- [25] Ravi Teja Mullapudi, Andrew Adams, Dillon Sharlet, Jonathan Ragan-Kelley, and Kayvon Fatahalian. 2016. Automatically Scheduling Halide Image Processing Pipelines. *ACM Trans. Graph.* 35, 4, Article 83 (July 2016), 11 pages. <https://doi.org/10.1145/2897824.2925952>
- [26] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. 2021. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '21)* (St. Louis, Missouri) (SC '21). Association for Computing Machinery, New York, NY, USA, Article 58, 15 pages. <https://doi.org/10.1145/3458817.3476209>
- [27] Jun-Seok Park, Jun-Woo Jang, Heonsoo Lee, Dongwoo Lee, Sehwan Lee, Hanwoong Jung, Seungwon Lee, Suknam Kwon, Kyungah Jeong, Joon-Ho Song, SukHwan Lim, and Inyup Kang. 2021. 9.5 A 6K-MAC

- Feature-Map-Sparsity-Aware Neural Processing Unit in 5nm Flagship Mobile SoC. In *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 64. 152–154. <https://doi.org/10.1109/ISSCC42613.2021.9365928>
- [28] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. ZeRO: Memory Optimizations toward Training Trillion Parameter Models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '20)* (Atlanta, Georgia) (SC '20). IEEE Press, Article 20, 16 pages.
- [29] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. 2021. ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning. <https://doi.org/10.48550/ARXIV.2104.07857>
- [30] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, Maximilien Breughe, Mark Charlebois, William Chou, Ramesh Chukka, Cody Coleman, Sam Davis, Pan Deng, Greg Diamos, Jared Duke, Dave Fick, J. Scott Gardner, Itay Hubara, Sachin Isgunji, Thomas B. Jablin, Jeff Jiao, Tom St. John, Pankaj Kanwar, David Lee, Jeffery Liao, Anton Lokhmotov, Francisco Massa, Peng Meng, Paulius Micikevicius, Colin Osborne, Gennady Pekhimenko, Arun Tejusve Raghunath Rajan, Dilip Sequeira, Ashish Sirasao, Fei Sun, Hanlin Tang, Michael Thomson, Frank Wei, Ephrem Wu, Lingjie Xu, Koichi Yamada, Bing Yu, George Yuan, Aaron Zhong, Peizhao Zhang, and Yuchen Zhou. 2020. MLPerf Inference Benchmark. , 446–459 pages. <https://doi.org/10.1109/ISCA45697.2020.00045>
- [31] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. 2015. U-Net: Convolutional Neural Networks for Biomedical Image Segmentation. arXiv:1505.04597 [cs.CV]
- [32] Samsung. 2021. Exynos 2100. <https://www.samsung.com/semiconductor/minisite/exynos/products/mobileprocessor/exynos-2100/>
- [33] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. 2019. MobileNetV2: Inverted Residuals and Linear Bottlenecks. arXiv:1801.04381 [cs.CV]
- [34] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. <https://doi.org/10.48550/ARXIV.1909.08053>
- [35] Rafael Sousa, Byungmin Jung, Jaehwa Kwak, Michael Frank, and Guido Araujo. 2021. Efficient Tensor Slicing for Multicore NPUs using Memory Burst Modeling. In *2021 IEEE 33rd International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*. 84–93. <https://doi.org/10.1109/SBAC-PAD53543.2021.00020>
- [36] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. 2015. Rethinking the Inception Architecture for Computer Vision. arXiv:1512.00567 [cs.CV]
- [37] Guangming Tan, Linchuan Li, Sean Trichele, Everett Phillips, Yungang Bao, and Ninghui Sun. 2011. Fast Implementation of DGEMM on Fermi GPU. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis* (Seattle, Washington) (SC '11). Association for Computing Machinery, New York, NY, USA, Article 35, 11 pages. <https://doi.org/10.1145/2063384.2063431>
- [38] Sanket Tavarageri, Alexander Heinecke, Sasikanth Avancha, Gagandeep Goyal, Ramakrishna Upadrasta, and Bharat Kaul. 2020. PolyDL: Polyhedral Optimizations for Creation of High Performance DL primitives.
- [39] Colin Unger, Zhihao Jia, Wei Wu, Sina Lin, Mandeep Baines, Carlos Efrain Quintero Narvaez, Vinay Ramakrishnaiah, Nirmal Prajapati, Pat McCormick, Jamaludin Mohd-Yusof, Xi Luo, Dheevatsa Mudigere, Jongsoo Park, Misha Smelyanskiy, and Alex Aiken. 2022. Unity: Accelerating DNN Training Through Joint Optimization of Algebraic Transformations and Parallelization. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, 267–284. <https://www.usenix.org/conference/osdi22/presentation/unger>
- [40] Hengjie Wang and Aparna Chandramowlishwaran. 2020. Pencil: A Pipelined Algorithm for Distributed Stencils. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–16. <https://doi.org/10.1109/SC41405.2020.00089>
- [41] Yunyang Xiong, Hanxiao Liu, Suyog Gupta, Berkin Akin, Gabriel Bender, Yongzhe Wang, Pieter-Jan Kindermans, Mingxing Tan, Vikas Singh, and Bo Chen. 2021. MobileDets: Searching for object detection architectures for mobile accelerators. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 3825–3834.
- [42] Yuanzhong Xu, HyoukJoong Lee, Dehao Chen, Blake Hechtman, Yanping Huang, Rahul Joshi, Maxim Krikun, Dmitry Lepikhin, Andy Ly, Marcello Maggioni, Ruoming Pang, Noam Shazeer, Shibo Wang, Tao Wang, Yonghui Wu, and Zhifeng Chen. 2021. GSPMD: General and Scalable Parallelization for ML Computation Graphs. <https://doi.org/10.48550/ARXIV.2105.04663>
- [43] Shijin Zhang, Zidong Du, Lei Zhang, Huiying Lan, Shaoli Liu, Ling Li, Qi Guo, Tianshi Chen, and Yunji Chen. 2016. Cambricon-x: An accelerator for sparse neural networks. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1–12.
- [44] S. Zhang, S. Zhang, Z. Qian, J. Wu, Y. Jin, and S. Lu. 2021. DeepSlicing: Collaborative and Adaptive CNN Inference With Low Latency. *IEEE Transactions on Parallel and Distributed Systems* 32, 9 (2021), 2175–2187. <https://doi.org/10.1109/TPDS.2021.3058532>
- [45] Sai Qian Zhang, Jieyu Lin, and Qi Zhang. 2020. Adaptive Distributed Convolutional Neural Network Inference at the Network Edge with ADCNN. In *49th International Conference on Parallel Processing - ICPP* (Edmonton, AB, Canada) (ICPP '20). Association for Computing Machinery, New York, NY, USA, Article 10, 11 pages. <https://doi.org/10.1145/3404397.3404473>
- [46] Z. Zhao, K. M. Barijough, and A. Gerstlauer. 2018. DeepThings: Distributed Adaptive Deep Learning Inference on Resource-Constrained IoT Edge Clusters. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 37, 11 (2018), 2348–2359. <https://doi.org/10.1109/TCAD.2018.2858384>
- [47] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, et al. 2020. Ansor: Generating high-performance tensor programs for deep learning. In *14th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 20)*. 863–879.
- [48] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P. Xing, Joseph E. Gonzalez, and Ion Stoica. 2022. Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, 559–578. <https://www.usenix.org/conference/osdi22/presentation/zheng-lianmin>
- [49] Shixuan Zheng, Xianjue Zhang, Leibo Liu, Shaojun Wei, and Shouyi Yin. 2022. Atomic Dataflow based Graph-Level Workload Orchestration for Scalable DNN Accelerators. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 475–489. <https://doi.org/10.1109/HPCA53966.2022.00042>
- [50] Li Zhou, Mohammad Hossein Samavatian, Anys Bacha, Saikat Majumdar, and Radu Teodorescu. 2019. Adaptive Parallel Execution of Deep Neural Networks on Heterogeneous Edge Devices. In *Proceedings of the 4th ACM/IEEE Symposium on Edge Computing* (Arlington, Virginia) (SEC '19). Association for Computing Machinery, New York, NY, USA, 195–208. <https://doi.org/10.1145/3318216.3363312>
- [51] Hongyu Zhu, Ruofan Wu, Yijia Diao, Shanbin Ke, Haoyu Li, Chen Zhang, Jilong Xue, Lingxiao Ma, Yuqing Xia, Wei Cui, Fan Yang, Mao Yang, Lidong Zhou, Asaf Cidon, and Gennady Pekhimenko. 2022.

ROLLER: Fast and Efficient Tensor Compilation for Deep Learning.
In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, 233–248.

<https://www.usenix.org/conference/osdi22/presentation/zhu>

Received 2022-09-02; accepted 2022-11-07